CCTV-BASED FIRE DETECTION SYSTEM

TEST STRATEGY

Document ID	QA-02
Project	CCTV-Based Fire Detection System (CCFDS)
Prepared By	Jayashini Jayaweera
Date	04-Jul-2026

1. Purpose

This Test Strategy defines the standard testing approach, methods, tools, and quality benchmarks applicable across all releases of the CCTV-Based Fire Detection System (CCFDS), independent of any single release's Test Plan.

2. Testing Types & Approach

Testing Type	Approach
Functional Testing	Validate detection triggers, alert workflows, dashboard actions and configuration screens against requirements
Detection Accuracy Testing	Run curated video/image datasets (fire, smoke, false-positive triggers like sunlight/steam/red objects) through the model and measure precision/recall
Integration Testing	Validate ONVIF/RTSP camera integration, NVR integration, SMS/Email/Push gateway, and fire panel relay/API integration
Regression Testing	Automated smoke + regression suite executed on every build and every AI model retraining
Performance Testing	Simulate concurrent multi-camera streams (e.g., 4, 16, 64, 128 feeds) to measure detection latency and system stability
Security Testing	Video stream encryption, authentication/authorization, API penetration testing, RTSP stream hijack prevention
Usability Testing	Control-room dashboard clarity, alert visibility, alarm acknowledgment workflow
Reliability/Endurance Testing	24-72 hour continuous run to validate memory leaks, stream drops, and alert consistency

Testing Type	Approach
Compatibility Testing	Multiple camera brands, resolutions (720p/1080p/4K), and network conditions (LAN/VPN/4G backhaul)

3. Test Levels

- Unit Testing — done by developers on detection modules, API endpoints, notification services.
- Integration Testing — validates module-to-module and third-party system interfaces.
- System Testing — full end-to-end flow: camera feed → detection → alert → dashboard & Alert Section → alert acknowledgement → incident resolve
- Acceptance Testing — business/control-room users validate operational readiness.

4. Tools & Technology

Category	Tool(s)
Test Management	Jira / Zephyr Scale
Functional Automation (UI)	Selenium WebDriver / Playwright
API Testing	Postman / RestAssured
AI Model Accuracy Validation	Python (OpenCV, scikit-learn) custom accuracy harness
Performance/Load Testing	Apache JMeter / Locust + custom RTSP stream simulator
Security Testing	OWASP ZAP, Burp Suite, Nmap
CI/CD Integration	Jenkins / GitHub Actions
Defect Tracking	Jira

5. Test Data Strategy

- Maintain a curated, versioned video/image dataset library: real fire, real smoke, near-miss (steam, fog, dust, red/orange objects, sunset glare), and normal scenes.
- Segregate datasets into Training-excluded validation sets to avoid data leakage into QA benchmarking.
- Refresh datasets quarterly and after every reported field false-positive/false-negative incident.

6. Defect Management

- Severity levels: Critical (missed fire detection / system down), High (delayed alert, false suppression), Medium (UI/reporting issue), Low (cosmetic).
- All Critical defects require root-cause analysis before closure.
- SLA: Critical – 4 hrs, High – 1 business day, Medium – 3 business days, Low – next release.

7. Entry/Exit Philosophy

Every release must meet a minimum detection accuracy benchmark and pass the automated regression suite before promotion to UAT, regardless of feature scope, to protect the life-safety nature of the product.

8. Metrics Tracked

Metric	Target
True Positive Detection Rate	≥ 95%
False Positive Rate	≤ 2%
Alert Latency (detection to notification)	≤ 5 seconds
System Uptime during endurance test	≥ 99.5%
Test Case Pass Rate at release	≥ 95%
Critical/High Defect Escape Rate	0 to Production

9. Roles Involved in Strategy Governance

- QA Lead — owns and reviews this strategy annually or after major architecture change.
- Product Owner — approves accuracy/latency thresholds given life-safety implications.
- Compliance/Safety Officer — reviews alignment with local fire-safety regulatory expectations.